

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT
TP. HỒ CHÍ MINH
Khoa Điện - Điện tử
Bộ môn Điện tử - Thông tin

Thực hành Học máy và Trí tuệ nhân tạo

Bài 6: Semantic Segmentation

Giảng viên: TS. Huỳnh Thế Thiện & PGS. TS Trương Ngọc Sơn

Tháng 03, 2026

1 Mục tiêu

Sau khi hoàn thành bài thực hành này, sinh viên có thể:

- Hiểu bài toán semantic segmentation
- Xây dựng pipeline segmentation bằng Deep Learning
- Huấn luyện mô hình U-Net
- Đánh giá mô hình bằng IoU và Dice Score
- Thực hiện dự đoán segmentation trên ảnh mới

2 Giới thiệu

Semantic segmentation là bài toán gán nhãn cho từng pixel trong ảnh.

Khác với classification hoặc object detection, segmentation cho phép xác định chính xác vùng của từng đối tượng.

Ứng dụng của semantic segmentation:

- xe tự lái
- chẩn đoán ảnh y khoa
- phân tích ảnh vệ tinh

- robot thị giác máy

3 Dataset

Trong bài thực hành này có thể sử dụng các dataset sau:

- Oxford Pet Dataset
- Carvana Image Masking Dataset
- Cityscapes Dataset

Mỗi dataset gồm:

- ảnh gốc
- segmentation mask

Mask là ảnh trong đó mỗi pixel thể hiện lớp của đối tượng.

Ví dụ:

- 0 = background
- 1 = object

4 Cấu trúc thư mục Dataset

Ví dụ cấu trúc dataset:

```
dataset/  
  images/  
    img1.jpg  
    img2.jpg  
    img3.jpg  
  
  masks/  
    img1.png  
    img2.png  
    img3.png
```

Ảnh và mask phải có cùng tên.

5 Train / Validation Split

Trước khi huấn luyện, cần chia dataset thành:

- training set
- validation set

Ví dụ sử dụng sklearn:

```
from sklearn.model_selection import train_test_split

train_images, val_images, train_masks, val_masks =
    train_test_split(

        image_paths,
        mask_paths,
        test_size=0.2,
        random_state=42
    )
```

6 Data Preprocessing

Trước khi đưa vào mô hình cần chuẩn hóa dữ liệu.

Các bước preprocessing:

- resize ảnh
- chuyển sang tensor
- chuẩn hóa pixel

```
from torchvision import transforms

image_transform = transforms.Compose([

    transforms.ToPILImage(),
    transforms.Resize((256,256)),
    transforms.ToTensor()
])
```

Transform cho mask:

```
mask_transform = transforms.Compose([
```

```

        transforms.ToPILImage(),
        transforms.Resize((256,256)),
        transforms.ToTensor()
    ])
]

```

7 Dataset Class

Ví dụ xây dựng Dataset class trong PyTorch.

```

import cv2
from torch.utils.data import Dataset

class SegmentationDataset(Dataset):

    def __init__(self, image_paths, mask_paths):

        self.image_paths = image_paths
        self.mask_paths = mask_paths

    def __len__(self):

        return len(self.image_paths)

    def __getitem__(self, idx):

        image = cv2.imread(self.image_paths[idx])
        mask = cv2.imread(self.mask_paths[idx],0)

        image = image_transform(image)
        mask = mask_transform(mask)

        return image, mask

```

8 DataLoader

Sau khi tạo Dataset, cần tạo DataLoader.

```

from torch.utils.data import DataLoader

train_loader = DataLoader(

```

```

        train_dataset,
        batch_size=8,
        shuffle=True
    )

    val_loader = DataLoader(

        val_dataset,
        batch_size=8
    )

```

9 Kiến trúc U-Net

U-Net là kiến trúc phổ biến cho semantic segmentation.

Mô hình gồm hai phần:

- Encoder (trích xuất đặc trưng)
- Decoder (khôi phục độ phân giải)

Skip connections giúp giữ lại thông tin không gian giữa encoder và decoder.

10 Ví dụ mô hình U-Net đơn giản

```

import torch
import torch.nn as nn

class SimpleUNet(nn.Module):

    def __init__(self):

        super(SimpleUNet, self).__init__()

        self.encoder = nn.Sequential(

            nn.Conv2d(3, 16, 3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2),

            nn.Conv2d(16, 32, 3, padding=1),

```

```

        nn.ReLU(),
        nn.MaxPool2d(2)
    )

    self.decoder = nn.Sequential(

        nn.ConvTranspose2d(32,16,2,stride=2),
        nn.ReLU(),

        nn.ConvTranspose2d(16,1,2,stride=2),
        nn.Sigmoid()
    )

    def forward(self,x):

        x = self.encoder(x)
        x = self.decoder(x)

        return x

```

11 Loss Function

Các loss phổ biến cho segmentation:

- Binary Cross Entropy
- Dice Loss

Ví dụ BCE:

```
criterion = nn.BCELoss()
```

Ví dụ Dice Loss:

```

def dice_loss(pred, target):

    smooth = 1

    intersection = (pred * target).sum()

    dice = (2*intersection + smooth) / \
        (pred.sum() + target.sum() + smooth)

```

```
return 1 - dice
```

12 Training Pipeline

Ví dụ pipeline huấn luyện:

```
device = torch.device("cuda" if torch.cuda.is_available()
                      else "cpu")

model = SimpleUNet().to(device)

optimizer = torch.optim.Adam(model.parameters(), lr
                              =0.001)

for epoch in range(10):

    model.train()

    for images, masks in train_loader:

        images = images.to(device)
        masks = masks.to(device)

        preds = model(images)

        loss = criterion(preds, masks)

        optimizer.zero_grad()

        loss.backward()

        optimizer.step()
```

13 Evaluation Metrics

Hai metric phổ biến:

- Intersection over Union (IoU)
- Dice Score

Công thức IoU:

$$IoU = \frac{Intersection}{Union}$$

14 IoU Calculation

```
def compute_iou(pred, mask):  
  
    pred = pred > 0.5  
  
    intersection = (pred & mask).sum()  
    union = (pred | mask).sum()  
  
    iou = intersection / union  
  
    return iou
```

15 Inference Pipeline

Ví dụ dự đoán trên ảnh mới.

```
image = cv2.imread("test.jpg")  
  
image = image_transform(image)  
  
image = image.unsqueeze(0)  
  
image = image.to(device)  
  
model.eval()  
  
with torch.no_grad():  
    pred = model(image)  
  
pred_mask = pred.squeeze().cpu().numpy()  
  
pred_mask = pred_mask > 0.5
```


16 Visualization

Hiển thị kết quả segmentation.

```
import matplotlib.pyplot as plt

plt.imshow(pred_mask, cmap="gray")
plt.show()
```

17 Error Analysis

Sau khi huấn luyện mô hình, cần phân tích các trường hợp dự đoán sai.

Một số nguyên nhân phổ biến:

- dataset nhỏ
- ảnh có nhiễu nhiều
- mô hình chưa đủ sâu
- overfitting

18 Bài tập

1. Tải một dataset segmentation từ Kaggle.

Gợi ý: có thể sử dụng các dataset như Oxford Pet Dataset hoặc Carvana Image Masking Dataset.

2. Khám phá cấu trúc dataset.

Gợi ý: kiểm tra các thư mục chứa ảnh và mask. Đảm bảo rằng mỗi ảnh có một mask tương ứng.

3. Hiển thị một số ảnh và mask.

Gợi ý: sử dụng thư viện matplotlib để hiển thị ảnh và mask tương ứng.

```
plt.imshow(image)
plt.imshow(mask)
```

4. Thực hiện train / validation split.

Gợi ý: sử dụng hàm `train_test_split` trong thư viện `sklearn` để chia dữ liệu.

5. Viết Dataset class cho PyTorch.

Gợi ý: lớp Dataset cần định nghĩa các hàm:

- `__init__()`
- `__len__()`
- `__getitem__()`

6. Tạo DataLoader cho training và validation.

Gợi ý: sử dụng `torch.utils.data.DataLoader` với batch size phù hợp (ví dụ 4 hoặc 8).

7. Huấn luyện mô hình U-Net.

Gợi ý: sử dụng optimizer Adam với learning rate ban đầu khoảng 0.001.

```
optimizer = torch.optim.Adam(  
    model.parameters(),  
    lr=0.001  
)
```

8. Thử nghiệm Dice Loss.

Gợi ý: so sánh kết quả khi sử dụng BCE Loss và Dice Loss.

9. Tính IoU trên tập validation.

Gợi ý: thực hiện dự đoán trên validation set và tính trung bình IoU của toàn bộ batch.

10. Hiển thị segmentation mask dự đoán.

Gợi ý: sử dụng matplotlib để hiển thị:

- ảnh gốc
- ground truth mask
- predicted mask

11. So sánh kết quả khi thay đổi learning rate.

Gợi ý: thử các giá trị:

- 0.01
- 0.001

- 0.0001

Quan sát sự thay đổi của loss và IoU.

12. Thử nghiệm các batch size khác nhau.

Gợi ý: thử batch size 4, 8 và 16 nếu GPU cho phép.

13. Phân tích các trường hợp mô hình dự đoán sai.

Gợi ý: tìm các ảnh mà IoU thấp và quan sát trực quan kết quả dự đoán.

14. Đề xuất cách cải thiện mô hình.

Gợi ý: có thể xem xét các phương pháp sau:

- tăng kích thước dataset
- data augmentation
- kiến trúc mạng sâu hơn
- thay đổi loss function

15. Thử nghiệm một kiến trúc segmentation khác.

Gợi ý: tìm hiểu các mô hình phổ biến như:

- DeepLabV3
- SegNet
- U-Net++